PartSight

Identifying marine diesel engine parts from a photo with AlexNet

Ziya Batur Koroglu

Mathematics of Data Science · Final Project · Fall 2026

A part request starts with a photo

A typical generator-parts request today:



1. Engineer sends a photo

Often no label, no part number, oily engine-room lighting



2. Supplier identifies it by eye

Depends on one experienced person being available



3. Look up the part & sources

Match engine model, find stock or a vendor



4. Send the quote

Every hour of delay keeps the ship's generator down

Step 2 is the bottleneck. PartSight automates it.

Why it matters



Shipping moves world trade

Over 80% of global trade by volume travels by sea (UNCTAD). Every vessel depends on its generators for power.



Downtime is expensive

A ship waiting on a part loses charter time and can miss port slots. Faster identification means faster quotes.



Expertise doesn't scale

Recognizing parts takes years of experience. New staff, nights and weekends create gaps.



Mistakes cost twice

A misidentified part means a wrong order, return shipping and a second delay.

Six high-volume parts the model learns to recognize



Fuel injector

Most frequently replaced wear part on generator engines



Turbocharger

High-value unit; fast ID speeds up big quotes



Piston

Overhaul item with many crown shapes and sizes



Connecting rod

Long forged part, easy to confuse with others



Crankshaft

Large, expensive, long lead time



Oil filter

High-volume consumable ordered constantly

Building the dataset

1

Collect

60 web image searches (10 per part) → 1,695 photos downloaded

2

Clean

Remove corrupt, tiny and near-duplicate photos

3

Filter with CLIP

A second AI model removes off-topic photos and labeled diagrams

4

Split

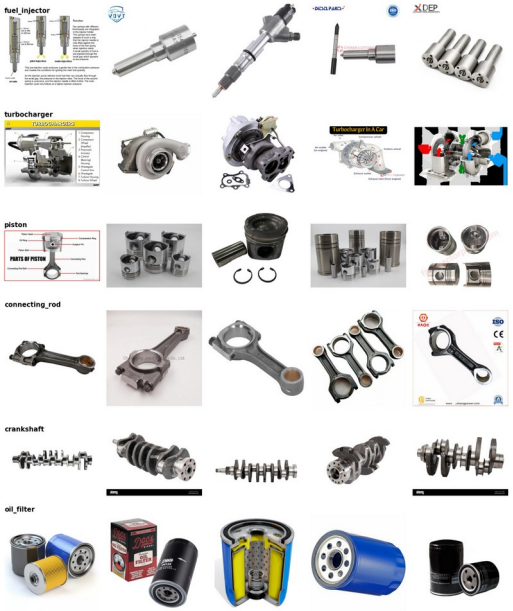
1,101 clean photos → 70% train · 15% validation · 15% test

What makes this hard

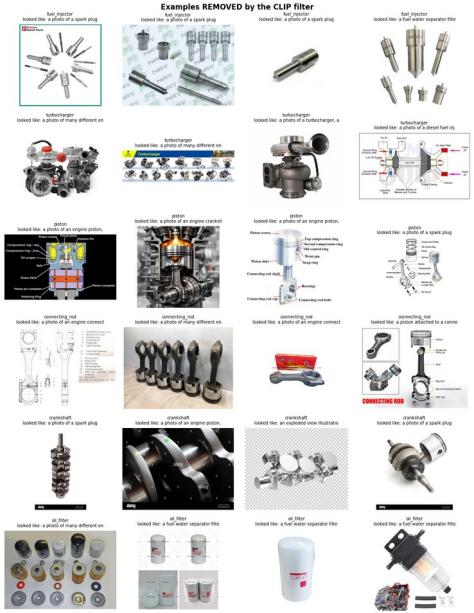
- Similar shapes: rods vs. crankshafts are both long forged steel
- Catalog photos on white vs. greasy field photos
- Lighting, angles and partial views vary a lot
- Small dataset: hundreds of images, not millions

Kept vs. removed photos

Kept: training photos



Removed by the CLIP filter



The filter is strict: it removes diagrams and junk, and some good photos too (nozzles mistaken for spark plugs). Losing a good photo costs less than training on a wrong one.

Goal: photo in, part type out

The task

Input: one color photo of a part

Output: the part type and a confidence for each of the 6 classes

Type: supervised multi-class image classification

How we judge success

Test accuracy

on images the model never saw in training

Per-class recall

no part type is systematically missed

Speed

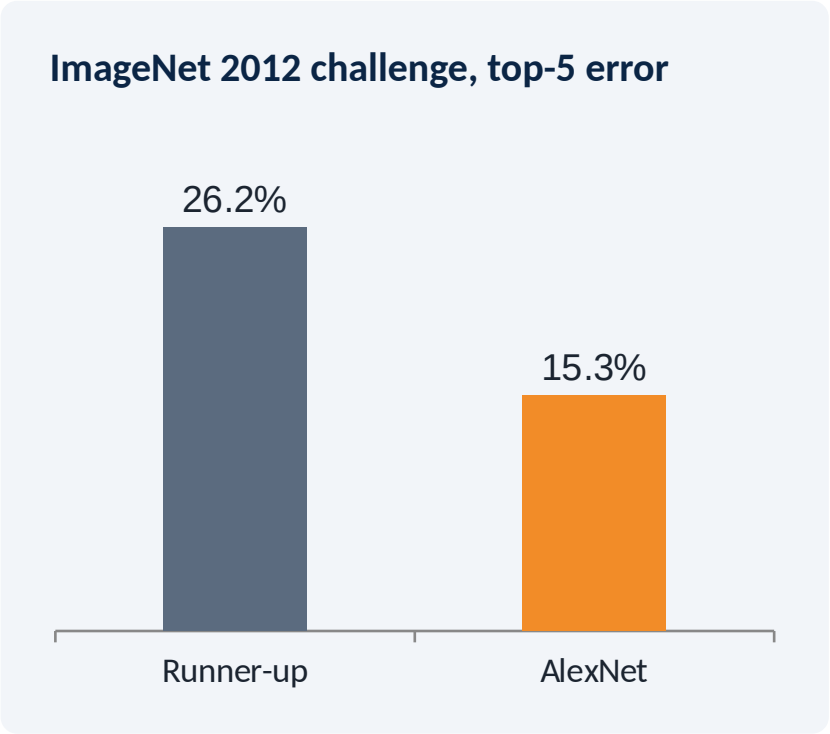
a prediction in well under a second



Part 2

Technical solution: AlexNet

AlexNet started the deep-learning era in vision



Krizhevsky, Sutskever & Hinton (2012)

Learns its own features

Earlier systems used hand-designed features. AlexNet learned edges, textures and shapes directly from pixels.

Deep + trained on GPUs

8 learned layers and 60 M parameters, trained on two GPUs — huge for 2012.

Why we use it

Simple enough to understand layer by layer, and pretrained weights let it learn from a small dataset like ours.

AlexNet architecture: 5 conv + 3 dense layers



Input	Conv1	Pool	Conv2	Pool	Conv3	Conv4	Conv5	Pool	FC6	FC7	FC8
3×224×224	64 @ 11×11, s4 → 55×55	3×3 max → 27×27	192 @ 5×5 → 27×27	→ 13×13	384 @ 3×3	256 @ 3×3	256 @ 3×3	→ 6×6 9,216 values	4096	4096	6 parts

Feature extractor — finds edges → textures → part shapes

Classifier — turns features into part scores (dropout 0.5)

ReLU after every conv and FC layer · ~57 M parameters

Layer by layer

Layer	Operation	Output	What it learns / does
Conv1	64 filters 11×11, stride 4 + ReLU	64×55×55	Edges, color blobs
MaxPool	3×3, stride 2	64×27×27	Shrinks maps, keeps strongest response
Conv2	192 filters 5×5 + ReLU	192×27×27	Corners, simple textures
MaxPool	3×3, stride 2	192×13×13	Down-sampling
Conv3–5	384 / 256 / 256 filters 3×3 + ReLU	256×13×13	Bolts, holes, cylinder & blade shapes
MaxPool	3×3, stride 2	256×6×6	Flattened to 9,216 numbers
FC6, FC7	Dense 4096 + ReLU + Dropout 0.5	4096	Combines features into part concepts
FC8	Dense 6 (replaced layer)	6	One score per part → softmax probabilities

Four ideas that made AlexNet work

ReLU activation

max(0, x) instead of tanh — trains ~6× faster because gradients don't vanish.

Dropout

Randomly switches off 50% of FC neurons during training, so the network can't memorize.

Overlapping max-pooling

Keeps the strongest signal in each region and makes the model tolerant to small shifts.

Data augmentation

Random crops, flips and color changes create new training images for free.

How our photos flow into the model

Raw photo	Resize	Augment	Normalize	Tensor	AlexNet	Softmax
any size, JPG/PNG	224×224 pixels	train only: crop, flip, ±15° rotate, color jitter	ImageNet mean & std per channel	3×224×224	6 output scores	e.g. 92% turbocharger



Transfer learning

We start from AlexNet weights already trained on 1.2 M ImageNet photos, so it already knows edges, metal textures and shapes. We replace only the last layer (4096 → 1000 classes becomes 4096 → 6 parts) and fine-tune the whole network on our images.

Training process

Loss
Cross-entropy

Optimizer
Adam

Learning rate
1×10^{-4}

Batch size
32

Epochs
15

Input
224×224 RGB

Framework
PyTorch on Colab

Tracking
Weights & Biases

Each epoch

1. Forward pass: images → predicted part probabilities
2. Cross-entropy loss measures how wrong the predictions are
3. Backpropagation computes gradients; Adam updates the weights
4. Score the validation set; keep the best epoch's weights
5. After training: one final score on the untouched test set

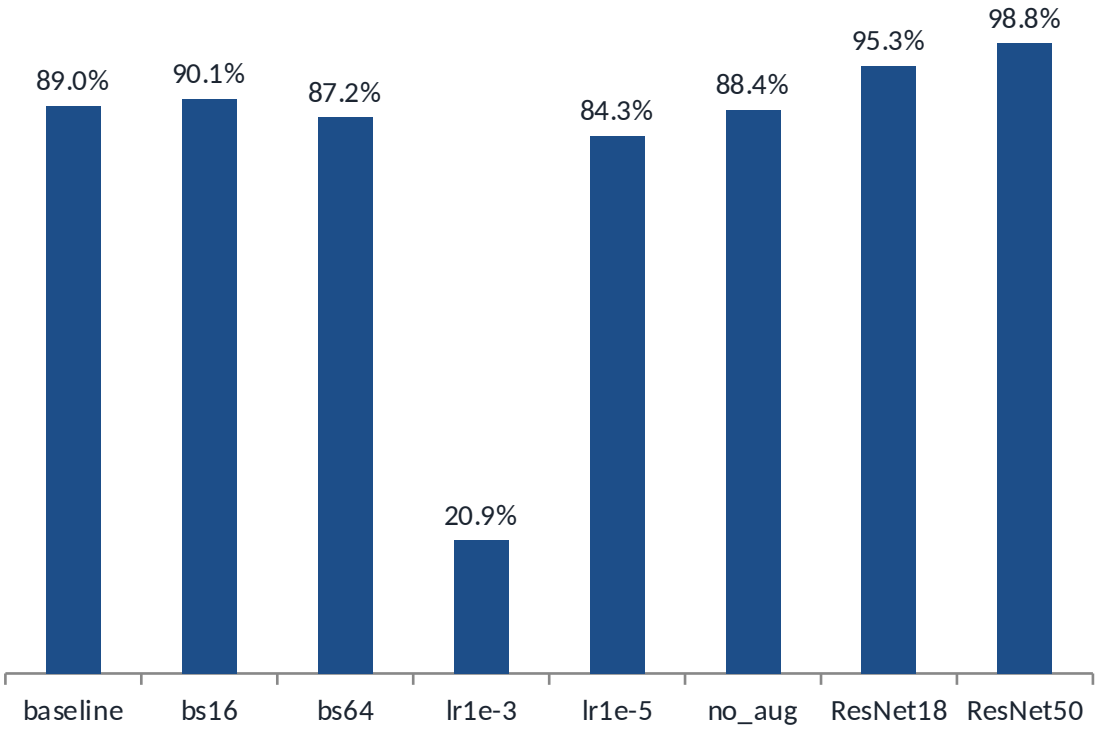
Experiments: change one thing at a time

Run	Model	Batch	Learning rate	Augment	Question
baseline	AlexNet	32	1e-4	Yes	Reference point
bs16 / bs64	AlexNet	**16 / 64**	1e-4	Yes	Do smaller or bigger batches help?
lr1e-3 / lr1e-5	AlexNet	32	**1e-3 / 1e-5**	Yes	Too fast vs. too slow learning?
no_aug	AlexNet	32	1e-4	**No**	Does augmentation reduce overfitting?
resnet18 / 50	**ResNet18/50**	32	1e-4	Yes	Does a modern network beat AlexNet?



Every run logs train/validation loss and accuracy per epoch, plus a test-set confusion matrix, to Weights & Biases.

Results: test accuracy by experiment



ResNet50: 98.8%

Best overall, on 172 photos it never saw

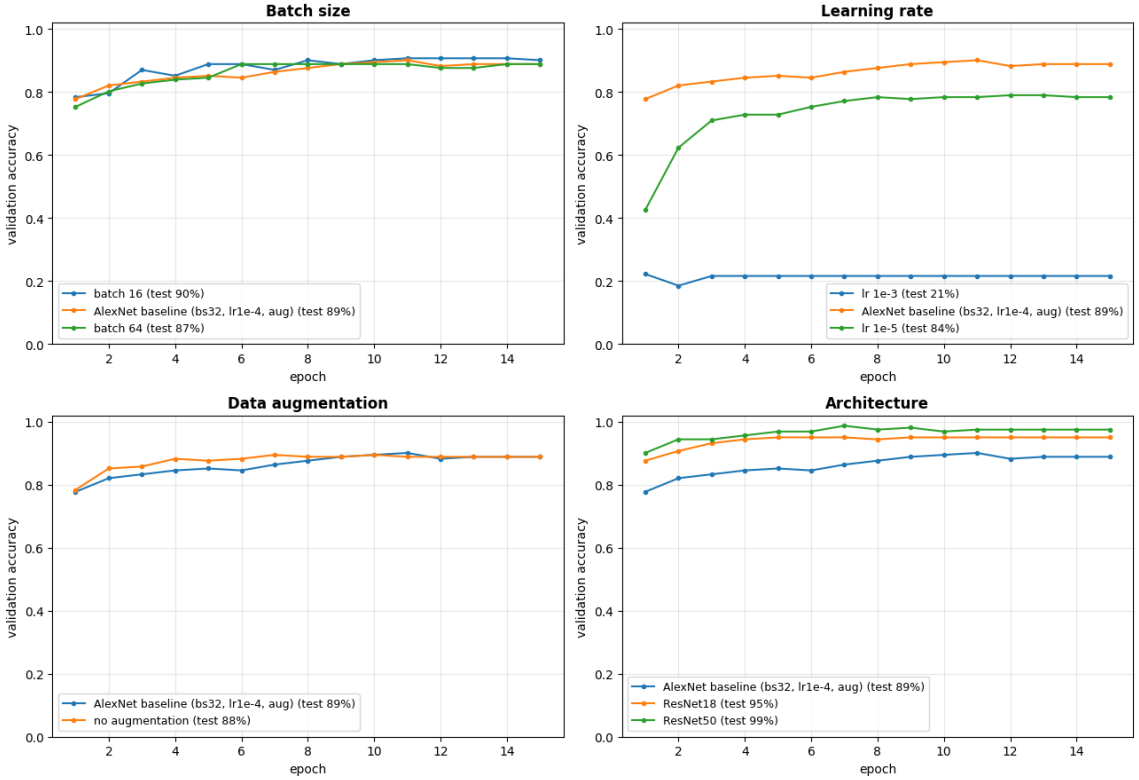
AlexNet: up to 90.1%

Best with batch size 16

LR 1e-3: 20.9%

Too high: wiped out the pretrained weights

What each experiment showed



Learning rate mattered most

1e-3 diverged; 1e-5 learned too slowly (84.3%)

Batch size: small effect

16 > 32 > 64, within 3 points

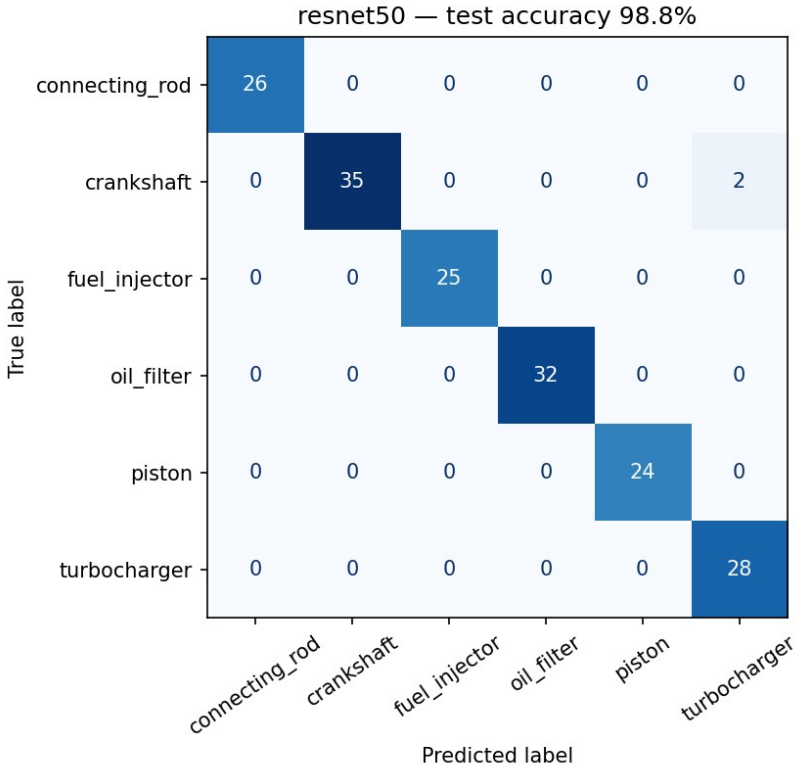
Augmentation slowed memorizing

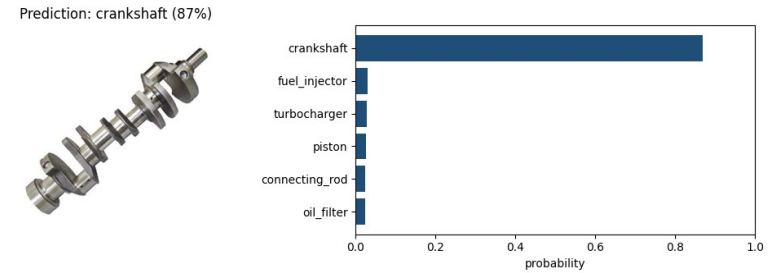
Small test gain: 89.0% vs 88.4%

Architecture: biggest jump

ResNets beat AlexNet in the same ~4 min

Best model: 2 mistakes out of 172 test photos





A new photo the model never saw: crankshaft, 87%

Both mistakes were crankshafts called turbochargers: a factory photo full of machinery and a whole-engine cutaway, not single-part photos. Below 60% confidence, PartSight answers “not sure” rather than guessing.

From photo to quote in seconds



Snap

Engineer photographs the part



Identify

PartSight predicts the part type



Match

Next: map to engine model & part number



Quote

Next: phone app for engineers

Best model: ResNet50, 98.8% · Best AlexNet: 90.1% · All runs tracked in Weights & Biases